Del CRUD local al CRUD consistente con backend

Cómo mantener coherencia entre IndexedDB y API

Dos fuentes de verdad

- IndexedDB
- Backend (.NET + PostgreSQL)

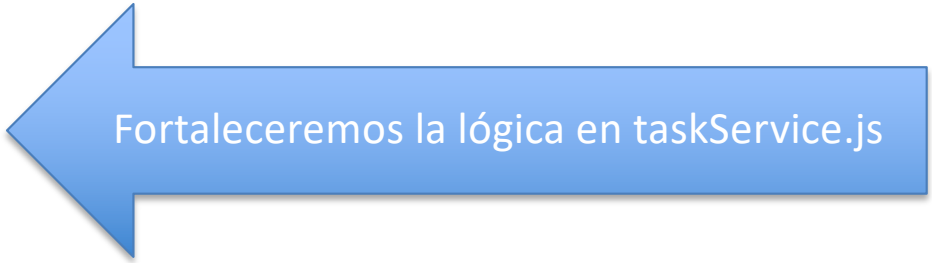
- ¿Qué pasa si ambas no representan exactamente la misma tarea?

Arquitectura actual

UI (uai.js)

↓

taskService.js



↓

db.js (IndexedDB)

↓

api.js (Backend)

Crear tarea no es trivial

- Flujo actual:
  - crear tarea local
  - enviar al backend
  - guardar respuesta del backend

- ¿Dónde puede aparecer un problema?

Flujo actual de creación

```
const localTask = {
  id: uid(),
  text,
  done: false,
  updatedAt: Date.now(),
};

await dbUpsertTask(localTask);

const serverTask = await apiCreateTask(localTask);
```

Solución conceptual propuesta

- crear tarea local temporal
- guardar localmente
- enviarla al backend
- recibir tarea real del servidor
- eliminar temporal local
- guardar definitiva local

Código corregido addTask

```
export async function addTask(text) {
  const localTask = {
    id: uid(),
    text,
    done: false,
    updatedAt: Date.now(),
  };

  await dbUpsertTask(localTask);

  try {
    const serverTask = await apiCreateTask(localTask);

    const normalized = {
      id: String(serverTask.id),
      text: serverTask.text,
      done: Boolean(serverTask.done),
      updatedAt: new Date(serverTask.updatedAt).getTime(),
      createdAt: serverTask.createdAt
        ? new Date(serverTask.createdAt).getTime()
        : Date.now(),
    };

    await dbDeleteTask(localTask.id);
    await dbUpsertTask(normalized);

    tasks = tasks.filter((t) => t.id !== localTask.id);
    tasks.unshift(normalized);
  } catch (error) {
    console.error("No se pudo guardar en backend", error);
    tasks.unshift(localTask);
  }

  return [...tasks];
}
```

Actualización (PUT)

- actualizar local
- guardar en IndexedDB
- si existe en backend → hacer PUT

Código clave PUT

```
if (!String(id).startsWith("t_")) {
  const serverTask = await apiUpdateTask(updated);
}
```

Código saveEdit

```
export async function saveEdit(id, text) {
  const task = tasks.find((t) => t.id === id);
  if (!task) return [...tasks];

  const updated = { ...task, text, updatedAt: Date.now() };

  tasks = tasks.map((t) => (t.id === id ? updated : t));
  await dbUpsertTask(updated);

  if (!String(id).startsWith("t_")) {
    try {
      const serverTask = await apiUpdateTask(updated);

      const normalized = {
        id: String(serverTask.id),
        text: serverTask.text,
        done: Boolean(serverTask.done),
        updatedAt: new Date(serverTask.updatedAt).getTime(),
        createdAt: serverTask.createdAt
          ? new Date(serverTask.createdAt).getTime()
          : updated.createdAt || Date.now(),
      };

      tasks = tasks.map((t) => (t.id === id ? normalized : t));
      await dbUpsertTask(normalized);
    } catch (error) {
      console.error("No se pudo actualizar en backend", error);
    }
  }

  sortTasks();
  return [...tasks];
}
```

Cambio de estado

- ¿Es una operación distinta o una actualización?

- Es una actualización (PUT)

Código toggleDone

```
export async function toggleDone(id) {
  const task = tasks.find((t) => t.id === id);
  if (!task) return [...tasks];

  const updated = { ...task, done: !task.done, updatedAt: Date.now() };

  tasks = tasks.map((t) => (t.id === id ? updated : t));
  await dbUpsertTask(updated);

  if (!String(id).startsWith("t_")) {
    try {
      const serverTask = await apiUpdateTask(updated);

      const normalized = {
        id: String(serverTask.id),
        text: serverTask.text,
        done: Boolean(serverTask.done),
        updatedAt: new Date(serverTask.updatedAt).getTime(),
        createdAt: serverTask.createdAt
          ? new Date(serverTask.createdAt).getTime()
          : updated.createdAt || Date.now(),
      };

      tasks = tasks.map((t) => (t.id === id ? normalized : t));
      await dbUpsertTask(normalized);
    } catch (error) {
      console.error("No se pudo actualizar estado en backend", error);
    }
  }

  sortTasks();
  return [...tasks];
}
```

Eliminación

- eliminar local
- eliminar en IndexedDB
- si existe en backend → DELETE

Código clave DELETE

```
if (!String(id).startsWith("t_")) {
  await apiDeleteTask(id);
}
```

Código removeTask

```
export async function removeTask(id) {
  tasks = tasks.filter((t) => t.id !== id);
  await dbDeleteTask(id);

  if (!String(id).startsWith("t_")) {
    try {
      await apiDeleteTask(id);
    } catch (error) {
      console.error("No se pudo eliminar en backend", error);
    }
  }

  return [...tasks];
}
```

Patrón de sincronización

acción del usuario

↓

actualización local

↓

persistencia local

↓

intento de sincronización con backend

↓

manejo de error

- manejo de ids temporales
- coherencia local + backend
- Aún falta sincronización robusta